

Navegação Autónoma na Formula Student: Uma Abordagem Baseada em Aprendizagem por Reforço

Luís Miguel Pereira Silva, Pedro Miguel S. A. Urbano dos Reis

Mestrado em Inteligência Artificial, Universidade do Minho, Braga, Portugal
{pg60390,pg59908}@alunos.uminho.pt

Abstract. Este documento apresenta o planeamento de um trabalho prático centrado no desenvolvimento de um sistema de condução autónoma para a competição Formula Student, recorrendo a técnicas de aprendizagem por reforço. O problema consiste em treinar um agente capaz de navegar numa pista delimitada por cones coloridos, respeitando as regras da categoria *Driverless*. Será desenvolvido um ambiente de simulação 2D com modelo cinemático de bicicleta, geração procedural de pistas e penalizações realistas. A metodologia assenta na implementação de um agente *Soft Actor-Critic* (SAC) para controlo contínuo, utilizando *Proximal Policy Optimization* (PPO) como *baseline*.

Keywords: Aprendizagem por Reforço · Condução Autónoma · Formula Student · Soft Actor-Critic · Simulação

1 Introdução e Definição do Problema

A competição Formula Student é a maior competição de engenharia automóvel universitária a nível mundial. A categoria *Driverless* desafia equipas a desenvolver veículos totalmente autónomos capazes de navegar numa pista delimitada por cones coloridos — azuis (esquerda), amarelos (direita) e laranja (partida e chegada) — sem intervenção humana [1]. O veículo deve detetar cones, planejar trajetórias e executar ações de direção e aceleração em tempo real.

A solução proposta assenta em aprendizagem por reforço [2]: treinar um agente que aprenda a conduzir maximizando o progresso e minimizando penalizações. Este relatório documenta a metodologia planeada, incluindo ambiente, algoritmos, ferramentas e validação. O código encontra-se em: <https://github.com/pedroreis2468/AR>.

2 Ambiente de Simulação e Geração de Dados

O treino será conduzido inteiramente em simulação, com um ambiente desenvolvido de raiz seguindo a API *Gymnasium*.

Modelo do veículo. Modelo cinemático de bicicleta [3] referenciado ao centro de gravidade, calibrado para um carro típico (~ 250 kg, *wheelbase* de 1.55 m):

$\dot{x} = v \cos(\theta + \beta)$, $\dot{y} = v \sin(\theta + \beta)$, onde $\beta = \arctan(l_r \tan \delta / L)$ é o ângulo de deslizamento lateral. Inclui limite de aceleração combinada ao atrito ($\mu \cdot g$).

Geração procedural e percepção. As pistas são geradas a partir de pontos de controlo numa elipse perturbada, suavizados com *splines* cúbicas periódicas, randomizando a cada episódio. A percepção é simulada por um sensor de cones (FOV: 180°, alcance: 15 m) que retorna os 3 cones mais próximos de cada lado com ruído gaussiano ($\sim 5\%$).

Regras *Driverless*. Cones derrubados: +2 s por cone; Cones laranja: +4 s por cone; Terminação: se ≥ 10 cones derrubados, desvio > 5 m do *centerline*, ou *off-course* > 2 s consecutivos.

3 Metodologia e Algoritmos

O problema é formulado como um MDP com $\gamma = 0.99$:

- **Observação** ($\mathbf{s} \in \mathbb{R}^{20}$): estado do veículo (5 dims), posições relativas dos cones (12 dims), distância às fronteiras e erro de *heading* (3 dims).
- **Ação** ($\mathbf{a} \in [-1, 1]^2$): direção normalizada e aceleração.
- **Recompensa**: $r_t = w_p \cdot \Delta_{\text{prog}} - w_s \cdot |\Delta \delta| - w_l \cdot d_{\text{lat}}^2 - w_t \cdot \Delta t - P_{\text{cones}}$, combinando progresso, suavidade, penalidade lateral, custo temporal e penalizações por cones.

O algoritmo principal será o SAC [4], um método *off-policy* com maximização de entropia, implementado com *Gaussian actor*, *twin Q-networks* e temperatura α auto-ajustada. Como *baseline*, será utilizado o PPO [5] via Stable-Baselines3. Aplicar-se-á *domain randomization* [6,7] variando massa ($\pm 15\%$) e atrito ($\pm 20\%$).

4 Ferramentas e Arquitetura de Software

Desenvolvimento em Python (≥ 3.10) com: Gymnasium ≥ 0.29 (API do ambiente), PyTorch ≥ 2.0 (redes *Actor-Critic*), Stable-Baselines3 ≥ 2.1 (PPO), PyGame ≥ 2.5 (renderização 2D), TensorBoard (monitorização) e Git+GitHub (versionamento).

5 Estado Atual da Implementação

A simulação e o agente SAC estão implementados e funcionais: o ambiente com modelo cinemático, geração procedural, sensor de cones e regras completas; o agente SAC custom em PyTorch; *scripts* de treino para modo custom e SB3; e avaliação com renderização PyGame. O trabalho restante foca-se no treino extensivo, *reward shaping*, *domain randomization*, análise comparativa, e exploração de *sim-to-sim transfer* para o simulador 3D PacSim.

6 Protocolo de Validação

A validação estrutura-se em: (1) **Desempenho desportivo** – avaliação em pistas nunca vistas, reportando taxa de voltas completas, cones derrubados e velocidade média ($\bar{x} \pm \sigma$); (2) **Eficiência de aprendizagem** – *reward* cumulativa e *episode length* ao longo dos *steps*; (3) **Comparação SAC vs. PPO** – desempenho, *sample efficiency*, tempo de treino e impacto do *domain randomization*.

7 Cronograma

Table 1. Planeamento e datas de entrega.

Período	Tarefa
–30 abril	Ambiente, agente SAC e <i>pipeline</i> de treino
–30 abril	Entrega do relatório de planeamento
1–11 maio	Treino SAC + PPO e <i>reward shaping</i>
12–18 maio	Experiências de <i>domain randomization</i>
19–25 maio	Análise de resultados; exploração PacSim
26 maio–1 junho	Preparação da apresentação
2 junho	Apresentação e resultados preliminares
3–15 junho	Incorporação de feedback; redação do relatório
16 junho	Entrega do relatório final

References

1. Kabzan, J., et al.: AMZ Driverless: The full autonomous racing system. *Journal of Field Robotics* **37**(7), 1267–1294 (2020). <https://doi.org/10.1002/rob.21977>
2. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. 2nd edn. MIT Press, Cambridge, MA (2018). ISBN 978-0-262-03924-6
3. Kong, J., Pfeiffer, M., Schildbach, G., Borrelli, F.: Kinematic and dynamic vehicle models for autonomous driving control design. In: 2015 IEEE Intelligent Vehicles Symposium (IV), pp. 1094–1099. IEEE (2015). <https://doi.org/10.1109/IVS.2015.7225830>
4. Haarnoja, T., et al.: Soft Actor-Critic algorithms and applications. arXiv:1812.05905 (2018). <https://arxiv.org/abs/1812.05905>
5. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal Policy Optimization algorithms. arXiv:1707.06347 (2017). <https://arxiv.org/abs/1707.06347>
6. Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., Abbeel, P.: Domain randomization for transferring deep neural networks from simulation to the real world. In: 2017 IEEE/RSJ IROS, pp. 23–30 (2017). <https://doi.org/10.1109/IROS.2017.8202133>
7. Ulrich, F., Wehrli, T.: End-to-End deep reinforcement learning for autonomous racing dynamics. Bachelor Thesis, ZHAW School of Engineering (2024). <https://doi.org/10.21256/zhaw-32000>